

AI Academy, National AI Center

AI-ENG-110 Software Engineering - Final Project

Async Research Assistant

Team	Async Research Assistant Team	Topic	4
Members	Samir Həsənov; Elmir Əsgərov; Ramil Məmmədəliyev	Date	2026-09-18
Repository	github.com/Hasanov57/AI-Academy---SE---Final-Project---AI-Researcher		
Tag	v1.0-final	Term	Spring 2026

Executive Summary

We built a command-line research assistant that queries Wikipedia, arXiv and Tavily concurrently and produces a concise answer with numeric citations. The student-owned layer wraps the supplied `ai/` package without changing its interface. It adds typed configuration, PostgreSQL persistence, a TTL cache, retries, timeouts, rate limiting and structured logging. A bounded `asyncio` pipeline isolates failed sources so remaining evidence can still reach synthesis. The live benchmark measured 1.440 seconds sequentially and 0.544 seconds concurrently, a 2.64x speedup with application caching disabled, while sixty-four offline tests pass with 73.56 percent coverage. A real Gemini request incompatibility taught us to keep provider adaptations outside the supplied package; if starting again, we would enforce that boundary from the first commit and add semantic citation-entailment checks.

Contents

1	Project Overview	2
2	Architecture	3-4
3	Concurrency and Performance	5
4	Robustness and Error Handling	6
5	Storage and Caching	7
6	Testing	8
7	Deployment and Reproducibility	9
8	Limitations, Tools and Benchmark Appendix	10

1. Project Overview

1.1 Problem framing and scope

The user submits one research question. The application searches three different evidence channels in parallel, merges usable excerpts and asks the provided synthesizer to write one short answer. The response contains inline numeric markers, reference titles and URLs, source timings and warnings for upstream failures.

The required entry point is `python -m researcher ask "question"`. The `--sources wiki,arxiv` option restricts retrieval, while `--no-cache` forces a fresh run. JSON output supports saved artefacts and automated inspection. A Streamlit interface reuses the same Researcher use case for project-day interaction, while the required CLI remains the primary automation and grading entry point.

1.2 Provider choice and the AI module contract

Google Gemini `gemini-3.5-flash-lite` performs synthesis because the supplied adapter supports structured output and its free tier controls cost. Tavily supplies general web results. Wikipedia and arXiv require no credentials. PostgreSQL is local during development and accessed asynchronously through `asyncpg`.

Every external research call crosses the instructor-owned API: `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web` or `ai.synthesize`. Our composition root instantiates the supplied provider adapters with validated secrets. The student package does not call provider SDKs or source endpoints directly.

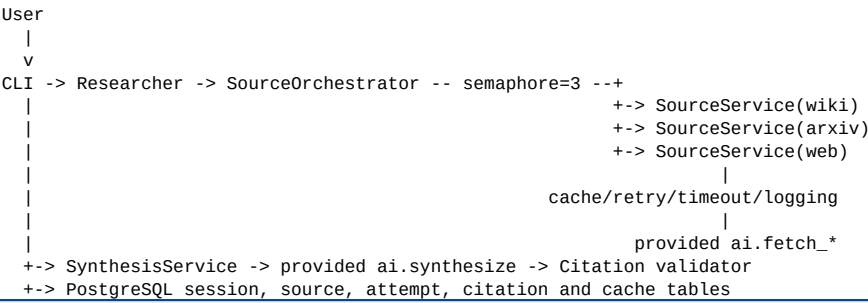
Inputs and outputs

Boundary	Validated input	Typed output
CLI	Question, source subset, cache flag	Exit code and rendered result
Core	ResearchRequest	ResearchResult
Source wrapper	SourceName and canonical query	SourceFetchOutcome
Provided AI	Question and Source list	AnswerWithCitations
Repository	ResearchResult or cache entry	Stored row or cache hit

Semantic defense: after the provided schema validation, the wrapper checks marker ranges, marker-to-reference equality and a citation on each substantive sentence. Invalid output triggers another synthesis attempt.

2. Architecture

2.1 Module map



The horizontal boundary above provided ai belongs to the team. The code below it remains unchanged. Network and LLM behaviour enter only through injected callables, making the boundary easy to mock.

2.2 Module-by-module responsibilities

Module	Owned responsibility
config.py	Typed environment configuration and live credential checks.
models.py	Pydantic contracts for requests, outcomes and results.
source_service.py	TTL cache, retry, timeout, rate-limit and call logging.
orchestrator.py	Shared HTTP client, bounded tasks and failure isolation.
synthesis_service.py	Thread offload, timeout, retry and citation validation.
researcher.py	Complete use case, URL de-duplication and persistence.
storage/	PostgreSQL implementation plus memory test substitute.
cli.py	Argument validation, result rendering and clean exit codes.

2.3 OOP design and data flow

Student-owned abstraction

ResearchRepository is an abstract base class because every runtime needs the same cache lifecycle, result transaction and resource cleanup operations. PostgresRepository implements durable production behaviour. MemoryRepository implements identical semantics for offline demonstrations and fast unit tests. The base class prevents the core use case from depending on asyncpg.

```
class ResearchRepository(ABC):
    async def get_cached_sources(source, query_key): ...
    async def put_cached_sources(source, query_key, sources, ttl): ...
    async def save_result(result): ...

class PostgresRepository(ResearchRepository): ...
class MemoryRepository(ResearchRepository): ...
```

Composition and pure functions

Researcher receives an orchestrator, synthesis service and repository through its constructor. SourceService receives a map of fetch callable. This composition is more appropriate than inheritance because these objects collaborate but do not represent subtypes of one another. Stateful behaviour lives in classes. Query canonicalization, URL normalization and de-duplication remain pure functions.

Data flow across boundaries

Step	Contract	Reason
1	ResearchRequest	Normalizes question and source subset.
2	SourceFetchOutcome	Carries results, cache status, time and error.
3	list[Source]	Uses the provided immutable schema without wrapping.
4	AnswerWithCitations	Preserves the instructor synthesis contract.
5	ResearchResult	Adds audit metadata and degradation warnings.

No naked dictionary crosses a service boundary. Dictionaries appear only while serializing JSONB or JSON artefacts.

3. Concurrency and Performance

3.1 Why asyncio

Wikipedia, arXiv and web search spend most of their time waiting for remote I/O. Native asyncio tasks therefore avoid the process startup and serialization costs of multiprocessing. The orchestrator shares one httpx connection pool and creates one task per selected origin. A configurable semaphore prevents unbounded work if the source set expands.

The supplied synthesis interface is synchronous. `asyncio.to_thread` moves that blocking SDK call away from the event loop without bypassing the provided adapter. This keeps the concurrency model honest: native async for HTTP and a thread only where the dependency forces it.

3.2 Sequential versus concurrent benchmark

Run	Sequential (s)	Concurrent (s)
1	1.440	0.535
2	1.439	0.544
3	1.523	0.677
Median	1.440	0.544

Speedup = $1.440 / 0.544 = 2.64x$. Three live retrieval runs used all sources on the same Windows machine with application caching disabled. The measurement isolates retrieval and excludes LLM synthesis; network and provider caches can still introduce normal variation.

3.3 Rate limit and politeness

- arXiv calls reserve a minimum one-second interval per process.
- A semaphore limits simultaneous sources to three by default.
- HTTP 429 and provider failures enter exponential retry up to three attempts.
- Per-source timing identifies the new bottleneck after parallelization.

4. Robustness and Error Handling

4.1 Wrapping the AI module

Concern	Implementation
Retry	Tenacity exponential wait; ProviderError, HTTP and timeout failures.
Timeout	Separate source and synthesis limits with asyncio.timeout.
Logging	INFO timing/count events, DEBUG payloads, WARNING source failures.
Rate limit	Semaphore, arXiv minimum interval and retry after provider 429.
Validation	Pydantic input plus semantic citation checks.
Degradation	Typed failed outcome; sibling tasks and synthesis continue.

4.2 Concrete failure exercise

A live Gemini request returned HTTP 400 because a provider-specific thinking option was incompatible with the selected model. The CLI showed a clean provider error and the logs identified the status without exposing the key. We restored the supplied adapter to the course version and added a student-owned ResearchLLM wrapper, preserving the instructor package boundary.

The incident corrected an early attempt to solve model compatibility inside the provided package. Moving the adaptation into student-owned code protected the course contract. Separate injected-failure tests verify that an exhausted source retry becomes a warning while sibling results still reach synthesis.

4.3 Input and output validation

- Questions are trimmed, whitespace-normalized and limited to 1,000 characters.
- At least one unique selector from wiki, arxiv or web is required.
- Missing live credentials fail before any external call.
- Citation markers must be present, in range, equal the returned reference set and attached to every substantive sentence.

5. Storage and Caching

5.1 PostgreSQL schema

Table	Source of truth	Important constraints
research_sessions	Question, answer, warnings, total time	UUID primary key
research_sources	Ordered title, URL and excerpt	Session FK plus ordinal
source_attempts	Cache flag, duration, error, count	Session plus origin key
citations	Marker to source-row mapping	Two-column foreign key
source_cache	Serialized Source list	Origin/query key and expiry index

One transaction inserts a completed session, its ordered sources, source attempt telemetry and citations. Cascading foreign keys prevent orphaned rows. Cache expiry and session creation timestamps have indexes because those are the main operational lookup paths.

5.2 Caching policy

The key combines SourceName with a canonical query produced by case-folding, trimming and collapsing whitespace. This makes equivalent capitalization and spacing hit the same entry. Each JSONB payload contains validated Source models. The default TTL is 86,400 seconds and remains configurable from the environment.

Expired rows are deleted lazily during lookup. A cache miss invokes the source and stores even an empty successful result, preventing repeated requests for a valid no-result query. The no-cache flag bypasses both reads and writes, which keeps benchmark runs comparable and lets users demand fresh evidence. A dedicated two-pass verification produced a 100 percent hit rate on its second pass.

Stored versus derived values

- Provider excerpts and final answers are stored as evidence of the exact run.
- Cache-hit status and timings are observed values stored per attempt.
- Speedup, source totals and rendered CLI text are derived and not stored.

6. Testing

6.1 Strategy and coverage

Suite	Count	Network	Result
Instructor AI smoke contract	Included	Mocked	Passing
Student-owned tests	Included	Replaced boundaries	Passing
Total	64	None	73.56% coverage

Coverage uses branch measurement over the researcher package. PostgreSQL error branches account for most uncovered lines because the local unit suite does not pretend a memory fake validates asyncpg protocol details. The documented Docker Compose smoke procedure is the final integration check.

6.2 Notable tests

- Happy path: one Wikipedia result reaches synthesis and persistence.
- Concurrency: three delayed fetchers overlap and beat the sequential wall clock.
- Degradation: arXiv raises while Wikipedia still produces a result.
- Retry: two transient failures precede a successful third call.
- Timeout: repeated deadline exhaustion surfaces after the retry budget.
- Citation output: missing, dangling and mismatched markers are rejected.
- Cache: canonical-equivalent queries hit and zero-TTL entries expire.

Quality commands

```
pytest --cov=researcher --cov-report=term-missing --cov-fail-under=60
ruff check src tests scripts
mypy src/researcher
python demo_ai.py --offline --limit 5
```

Ruff and strict mypy complete without issues. The instructor smoke tests remain unchanged and pass inside the same full suite.

7. Deployment and Reproducibility

7.1 Docker image

The last verified single-stage image was 515.4 MB. It uses Python 3.12 slim, installs fully pinned dependencies, copies the provided and student packages, and drops to a non-root user. Docker Compose runs PostgreSQL 17.6 Alpine and waits for pg_isready before the application starts.

```
docker build -t async-researcher .
docker compose up -d db
docker compose run --rm researcher

docker run --rm async-researcher python -m researcher ask \
  "What is photosynthesis?" --offline --storage memory
```

7.2 Configuration

Group	Variables
LLM choice	LLM_PROVIDER, LLM_MODEL
LLM keys	GOOGLE_API_KEY, OPENAI_API_KEY, ANTHROPIC_API_KEY
Web	WEB_SEARCH_PROVIDER, TAVILY_API_KEY, SERPER_API_KEY
Storage	DATABASE_URL, STORAGE_BACKEND, POSTGRES_PASSWORD, CACHE_TTL_SECONDS
Timeouts	PER_SOURCE_TIMEOUT_SECONDS, SYNTHESIS_TIMEOUT_SECONDS
Retries	MAX_RETRY_ATTEMPTS, RETRY_MIN_WAIT_SECONDS, RETRY_MAX_WAIT_SECONDS
Limits	MAX_PARALLEL_SOURCES, MAX_SOURCES_PER_QUERY, ARXIV_MIN_INTERVAL_SECONDS
Interface	LOG_LEVEL, QUESTION_MAX_LENGTH, HTTP_USER_AGENT

Secrets use Pydantic SecretStr, never appear in INFO logs and live only in the ignored .env file. A missing key for the selected live provider or a missing Compose password fails before work begins. Other settings have typed defaults; out-of-range or malformed values fail Pydantic validation with a clear message.

Reproducibility controls

- Every dependency uses an exact version.
- The package exposes both python -m researcher and a console script.
- Offline mode verifies installation without credentials or network.

8. Limitations and Future Work

- Citation syntax checks cannot prove semantic entailment.
- A single PostgreSQL instance has no replication or automated backup.
- In-process arXiv limiting does not coordinate multiple containers.
- Provider excerpts can omit context even when all systems are healthy.
- Provider failover, streaming answers and public hosting remain outside release 1.0; the included Streamlit UI runs through Docker Compose.

Tools and Acknowledgements

AI tools were used for architecture planning and technical assistance during development. The team reviewed and adapted the suggestions and verified the final system through the complete offline suite and live demonstrations. The provided ai package and course templates came from AI Academy.

References

- [1] AI Academy, AI-ENG-110 Final Project Brief, Spring 2026.
- [2] Gemini model documentation: ai.google.dev/gemini-api/docs
- [3] Tavily API documentation: docs.tavily.com
- [4] MediaWiki API documentation: mediawiki.org/wiki/API:Main_page
- [5] arXiv API documentation: info.arxiv.org/help/api/index.html
- [6] Python asyncio documentation: docs.python.org/3/library/asyncio.html

Appendix - Reproducing the benchmark

```
docker compose build researcher
docker compose up -d db
docker compose run --rm researcher python scripts/bench.py --runs 3
```

The live command requires the Gemini and Tavily keys in .env even though only source timing enters the benchmark composition root. Run both modes on the same machine with caches bypassed and record the raw table.

End of report